

Recherche Textuelle 1

Approche naïve

Vocabulaire :

- Motif / Slicing / Recherche d'un motif / Coût d'un algorithme / Algorithme naïf.

Prérequis :

- Bases de Python
- Notion de complexité d'un algorithme.
- Connaître le type *string*.

bnoujor, je crhehce de l
adie, mon caliver est
tmboé, les lretets se snot
mealneges

Objectifs :

- Revoir les bases sur les chaînes de caractères.
- Se familiariser avec le slicing.
- Implémenter l'algorithme naïf de recherche de motif dans un texte.
- Connaître et savoir retrouver la complexité temporelle de l'algorithme naïf de recherche d'un motif.



- Le problème de la recherche d'un motif dans un texte est omniprésent en informatique : éditeur de texte, moteur de recherche, analyse de séquences biologiques, imagerie informatique, composition musicale automatique, etc...
- Il suscite encore de nombreuses recherches, à la fois d'un point de vue pratique (pour optimiser les méthodes), et d'un point de vue théorique.

I- RECHERCHE D'UN CARACTÈRE DANS UNE CHAÎNE

A- Rappels sur les chaînes de caractères

Commençons par quelques rappels sur les chaînes de caractères dans le langage Python.

- Les chaînes de caractères sont des variables de type `string`.
- Une chaîne de caractères peut être écrite au choix entre apostrophes (caractère `'`) ou entre guillemets (caractère `"`). Ainsi on peut écrire indifféremment `'Voici une belle journée'` ou `"Voici une belle journée"`.
Nous utiliserons la seconde forme, plus répandue dans les autres langages de programmations.
- Pour affecter cette chaîne de caractère à une variable, par exemple nommée `s`, on écrit simplement : `s = "Voici une belle journée"`
- La longueur d'une chaîne `s` est obtenue avec `len(s)`. Les espaces sont considérés comme des caractères.
- Les caractères de la chaîne sont indicés à partir de 0.
Dans l'exemple précédent, le caractère `s[3]` correspond à la lettre `c`.
Le (i+1)-ième caractère est obtenu avec `s[i]`.
- Même si cela n'est pas au programme de Terminale, il est bon de connaître une technique très Pythonesque : le **slicing**. Cette technique qui est aussi utilisée pour les listes, permet de récupérer facilement des sous-chaînes d'une chaîne de caractères.
Soit un chaîne `s` telle que `s = "Bonjour"` alors :
`s[0:3]` correspond à la sous-chaînes `"Bon"` et peut aussi s'écrire `s[:3]`
`s[3:7]` correspond à la sous-chaînes `"jour"` et peut aussi s'écrire `s[3:]`
- Il est possible de comparer des chaînes avec l'opérateur `==`.
`s == "Bonsoir"` donne `False` comme résultat.

B- Recherche d'un motif dans un texte

- Nous allons nous intéresser à la recherche d'un **motif** dans un **texte**.
Il faut savoir qu'un motif ne se limite pas à une suite de caractères (ASCII ou pas). Il peut tout aussi bien s'agir d'une suite d'octets dans un fichier quelconque ou bien d'une séquence ADN (suites de bases ATGC).
- Plus généralement, on considère :
 - un alphabet A fini (nos 26 lettres par exemples)
 - deux suites ordonnées d'éléments de A (le texte et le motif).
- On cherche à répondre à différentes questions plus ou moins équivalentes :
 - Le motif apparaît-il dans le texte ?
 - Si oui, quelle est la première position où il apparaît ?
 - Quelles sont toutes les positions où ce motif apparaît ?
- On notera **T** la chaîne de caractères contenant le **texte** et **M** celle contenant le **motif**.

Exercice 1 : Recherche d'un motif et de sa position.

Si on recherche le motif M **abaa** dans le texte **aacabacabaabaa** :

T	a	a	c	a	b	a	c	a	b	a	a	b	a	a
indice	0	1	2	3	4	5	6	7	8	9	10	11	12	13

Combien de fois le motif est-il présent ? À quelle position ?

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

C- Recherche d'un caractère dans un texte

Le cas le plus simple est celui où le motif est un simple caractère. Il s'agit d'une recherche linéaire déjà vu en classe de première.

Exercice 2 :

- 1) Implémente une fonction **recherche_1caractere** qui renvoie **True** si le caractère est présent dans la chaîne passée en argument et **False** sinon.
- 2) Quel est le **coût** (=complexité temporelle) de l'algorithme de recherche d'un caractère ?

- 3) Implémente une fonction `recherche_position1caractere` qui renvoie l'indice de la première occurrence dans la chaîne.
- 4) Implémente une fonction `recherche_nbMots` qui renvoie le nombre de mots dans une phrase passée en argument.

[Recopie ton algorithme ici puis sauvegarde ensuite ton fichier pdf.]

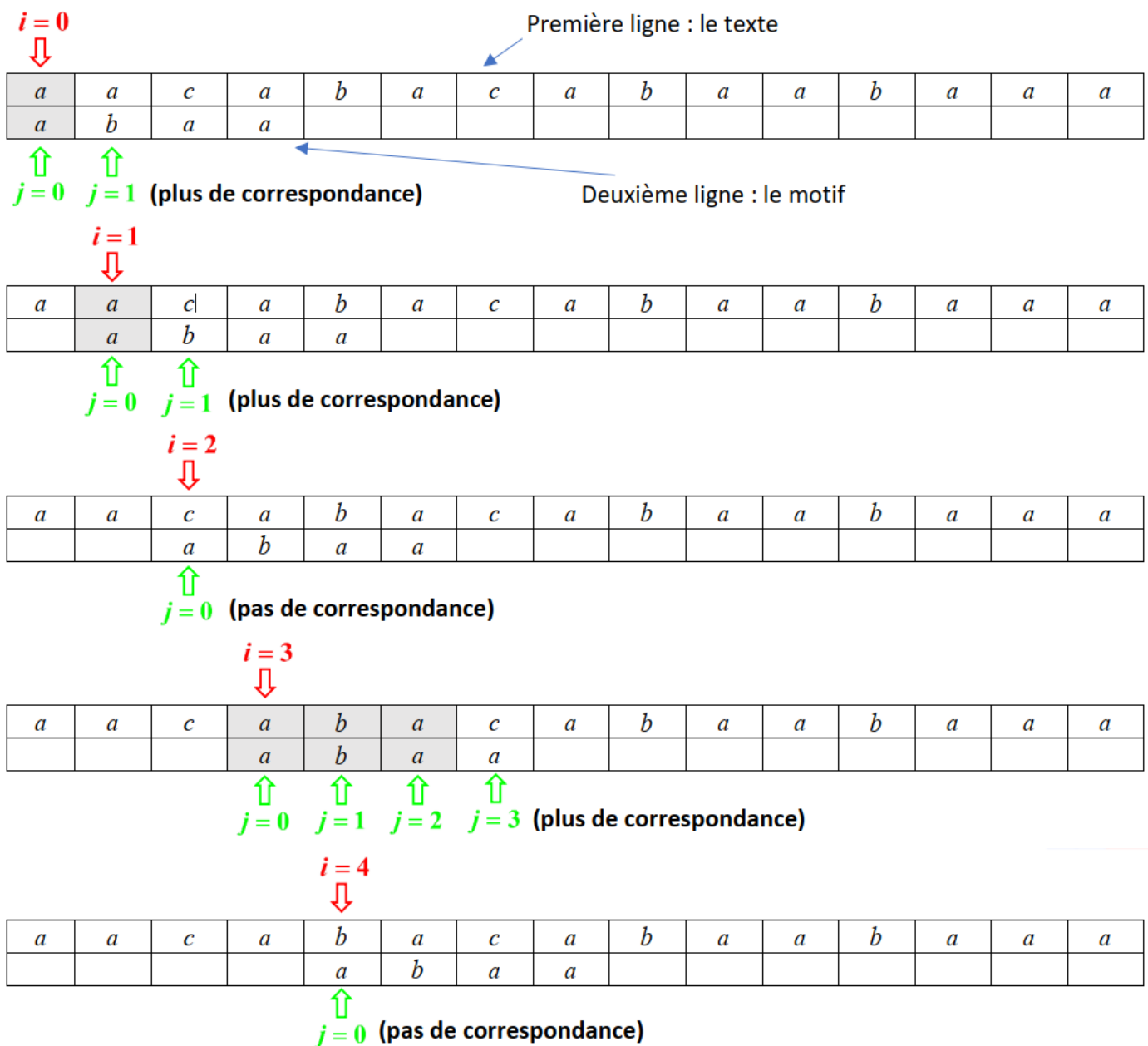
II- RECHERCHE NAÏVE D'UN MOTIF "long" DANS UNE CHAÎNE

- À présent, nous allons chercher un motif "long" dans un texte T. Cela signifie simplement que le motif est plus grand qu'un seul caractère.
- Commençons par une première version de recherche : l'**algorithme naïf** (dit aussi de force brute). On parle de recherche naïve car c'est l'une des premières idées à venir à l'esprit.

Exercice 3 : Propose un algorithme, en langage naturelle, pour vérifier qu'un mot se trouve dans un texte.

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

- L'idée est de réaliser une comparaison caractère après caractère du texte T et du motif M. On parcourt les caractères de T tant qu'ils sont différents du premier caractère de M. Dès qu'on trouve un caractère identique, on parcourt les caractères suivants tant qu'ils correspondent.
 - Si un caractère diffère alors qu'on n'a pas atteint la fin du motif, on reprend la recherche du premier caractère identique, à partir du caractère suivant dans le texte.
 - Si tous les caractères correspondent, on retourne la position du premier caractère du motif trouvé dans le texte.
 - Enfin, si aucune occurrence de la chaîne recherchée n'apparaît dans la chaîne initiale, l'algorithme se doit de le signaler, en retournant une valeur négative par exemple.

Exemple :

					$i=5$ ↓									
a	a	c	a	b	a	c	a	b	a	a	b	a	a	a
					a	b	a	a						
					↑ $j=0$	↑ $j=1$	(plus de correspondance)							
					$i=6$ ↓									
a	a	c	a	b	a	c	a	b	a	a	b	a	a	a
						a	b	a	a					
					↑ $j=0$	(pas de correspondance)								
					$i=7$ ↓									
a	a	c	a	b	a	c	a	b	a	a	b	a	a	a
							a	b	a	a				
							↑ $j=0$	↑ $j=1$	↑ $j=2$	↑ $j=3$	chaque lettre du motif correspond !			

- Ainsi, dans notre exemple, on signale l'occurrence du motif dans le texte en stockant l'indice $i = 7$ puis on poursuit la démarche afin de trouver d'autres occurrences du motif.

Exercice 4 : Implémentation de l'algorithme naïf

Il s'agit de coder l'algorithme permettant de renvoyer la liste des emplacements du motif dans un texte.

Exemple :

```
texte="aacabacabaabaa"
motif="abaa"
on obtient : [7, 10]
```

Pour les aventurier(e)s du code

À vous de jouer. Vous pouvez écrire la fonction en Python sur Pyzo.
Nommez la `recherche_naive`.



En cas de blocages, rendez-vous dans le cadre de droite.

Pour une version guidée

1. Soit n la longueur du texte et m la longueur du motif. Pourquoi n'est-il pas nécessaire de chercher le début du motif jusqu'à la place $n-1$? Jusqu'à quelle valeur de i faut-il parcourir le texte ?
2. On suppose que le premier caractère en place i est le même pour le texte et pour le motif. Quelles sont les deux conditions qui permettent de continuer à chercher la possibilité que le motif se trouve bien à cet endroit ?
3. Exprime les deux conditions précédentes en fonction de i , j et m .
4. À quelle condition considère-t-on que le motif est finalement bien présent à la place i .
5. Dans cette fonction de recherche, faut-il deux boucles for imbriquées ou bien une boucle while dans une boucle for ? Justifie.
6. Écrire la fonction Python, nommée `recherche_naive` (`texte`, `motif`) qui renvoie, sous la forme d'une liste, l'indice du premier caractère de chaque occurrence du motif trouvé dans le texte. Teste ta fonction pour finir.



[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

Exercice 5 : Quelle est la complexité de l'algorithme naïf ?

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

Exercice 6 : Petit exercice si vous êtes en avance. Il s'agit de modifier la fonction recherche_naïve en utilisant le slicing.

Aide : on ne modifie pas ce qui est à l'extérieur de la boucle for, ni la boucle for elle-même.

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

III- DES ÉCONOMIES POSSIBLES

- Il existe plusieurs façons d'économiser du temps de recherche dans la recherche d'un motif dans un texte.

Exercice 7 : Fais des propositions de pistes pour améliorer l'efficacité de l'algorithme naïf dans la recherche d'un motif.

Propose des améliorations possibles en langage naturelle ?

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]